

Sa FTP servera (folder **Download/RSII**, user/pass: **student_250250**) preuzmite template projekta na kome ćete raditi implementaciju funkcionalnosti opisanih u nastavku. Promijenite konekcijski string u fajlu *appsettings.Development.json* kako biste se povezali na SQL server na kojem ćete razvijati i testirati svoj projekt.

- **Adresa i naziv servera:** 192.168.0.1\\Exams
- **Port:** 1999
- **Baza podataka:** Koristeći migracije kreirajte bazu podataka sa vašim brojem indeksa npr. IB150051
- **Pristupni podaci:**
Username = john
Password = doe2025

Svi studenti ispit rade na **istoj instanci servera**, pa je važno da radite isključivo unutar baze koja pripada vašem projektu.

1. Pokrenite migraciju i kreirajte vašu bazu - pokrenite *Package Manager Console*, odaberite *Service* projekat, te izvršite komandu:

```
Update-Database
```

Nakon uspješne migracije, u bazi bi trebalo da postoje pristupni podaci:

```
username=customer1
```

```
password=Test123
```

Tokom implementacije zadataka opisanih u nastavku obavezno je poštovati postojeću strukturu i principe projekta što podrazumijeva dodavanje potrebnih (dto)entiteta, servisa i kontrolera u odgovarajuće slojeve projekta, a nikako instanciranje DbContext i drugih objekata (npr. na nivou kontrolera i sl.) gdje po strukturi projekta ne pripadaju.

2. U okviru templejta projekta, na Web API dijelu, implementirati funkcionalnost za razmjenu poruka između korisnika.

Dodati entitet **Chat**BrojIndeksa koji služi za čuvanje podataka o razgovorima između korisnika. Razgovor treba sadržavati: naziv, datum i vrijeme kreiranja, učesnike u razgovoru, te druge attribute koje smatrate potrebnim. U jednom razgovoru mogu učestvovati najviše dva različita korisnika. Između dva korisnika može postojati samo jedan razgovor, bez obzira na redoslijed u kojem su korisnici navedeni.

Dodati entitet **ChatMessage**BrojIndeksa koji služi za čuvanje poruka poslanih u okviru razgovora. Poruka treba sadržavati vezu na razgovor, vezu na korisnika koji je poslao poruku, sadržaj poruke, datum i vrijeme nastanka, te druge attribute koje smatrate potrebnim.

Sadržaj poruke je obavezan i ne smije sadržavati samo prazne znakove. Poruku može poslati samo korisnik koji učestvuje u odabranom razgovoru. Korisniku omogućiti brisanje samo vlastitih poruka.

Potrebno je dodati svu potrebnu infrastrukturu za upravljanje razgovorima i porukama: modele, bazne tabele, migracije, DTO/request/response modele, servise, kontrolere i validacije.

Prilikom učitavanja razgovora trenutno prijavljenog korisnika potrebno je vratiti osnovne podatke o razgovoru, podatke o drugom učesniku, datum kreiranja i posljednje poslanu poruku. Razgovore sortirati tako da se na vrhu nalazi razgovor u kojem je posljednja poruka najnovija. Razgovori bez poruka mogu se sortirati prema datumu kreiranja.

3. U Flutter dijelu projekta, na profilnoj formi dodati dugmić „Poruke“. Klikom na pomenuti dugmić korisnik se preusmjerava na formu za prikaz i upravljanje razgovorima (*chat_screen.dart*).

Na vrhu forme omogućiti unos naziva novog razgovora i odabir korisnika sa kojim trenutno prijavljeni korisnik želi započeti razgovor. Naziv razgovora je obavezan. U listi korisnika nije potrebno prikazivati trenutno prijavljenog korisnika.

Odabir korisnika treba ujedno koristiti kao filter liste razgovora. Ako korisnik nije odabran, prikazati sve razgovore trenutno prijavljenog korisnika. Nakon odabira korisnika prikazati samo razgovor sa tim korisnikom, ako postoji.

Dodati dugmić „Kreiraj razgovor“ kojim se kreira novi razgovor sa odabranim korisnikom. Ako razgovor između ta dva korisnika već postoji, ne treba kreirati novi razgovor. Potrebno je prikazati odgovarajuću poruku.

Novokreirani razgovor dodati u listu koja se nalazi ispod forme za unos, te za svaki razgovor prikazati: naziv razgovora, ime drugog korisnika, datum kreiranja, sadržaj posljednje poruke. Ako u razgovoru još nema poruka, umjesto posljednje poruke prikazati odgovarajući tekst, na primjer „Nema poruka“.

Klikom na razgovor otvoriti formu za prikaz poruka (*chat_details_screen.dart*).

4. Na formi *chat_details_screen.dart* prikazati sve poruke koje pripadaju odabranom razgovoru.

Poruke trenutno prijavljenog korisnika prikazati na desnoj strani forme, a poruke drugog korisnika na lijevoj strani. U okviru svake poruke prikazati sadržaj poruke i vrijeme njenog nastanka.

Na dnu forme omogućiti unos i slanje nove poruke. Slanje prazne poruke nije dozvoljeno. Nakon uspješnog slanja potrebno je očistiti polje za unos i osvježiti prikaz poruka.

Korisniku omogućiti brisanje vlastitih poruka. Opcija za brisanje ne treba biti dostupna za poruke koje je poslao drugi korisnik.

Nakon brisanja poruke potrebno je osvježiti prikaz poruka. Također je potrebno ažurirati prikaz posljednje poruke na formi chat_screen.dart. Ako je obrisana posljednja poruka u razgovoru, kao posljednju poruku prikazati prethodnu poruku ili tekst „Nema poruka“ ako razgovor više nema poruka.

5. Na kraju ispita, na backend projektu odaberite opciju Clean Solution (u okviru Visual Studio-a napravite desni klik na naziv solution-a), a unutar Flutter projekta, u terminalu Visual Studio Code-a, izvršite komandu `flutter clean`.

Nakon toga, zapakujte projekat (.zip ili .rar) u folder imenovan vašim brojem indeksa i postavite na FTP server (folder **Upload/RSII**, user/pass: **student_dm**)